**Iheb Belaid**
+216 95 827 073 | ihebbelaid3@gmail.com | linkedin.com/in/iheb-belaid/

# KAMKA IT Assessment Notes

*Cloud & DevOps Intern (CI/CD & Infrastructure) - Summer 2026*

**Repository:** kamka-cloud-devops-assessment

## Architecture Summary

This submission implements a minimal three-tier Todo application focused on delivery lifecycle and operational clarity rather than application complexity.

- frontend: static HTML, CSS, and JavaScript served by Nginx
- api: Spring Boot REST API exposing Todo endpoints and health checks
- db: PostgreSQL persistence layer
- monitoring: Uptime Kuma watching the running stack
- ci/cd: GitHub Actions building, testing, publishing, and triggering deployment

### Change flow from push to production:

1. A push triggers the GitHub Actions workflow.

2. The API test job runs first and fails loudly if the backend is broken.

3. If tests pass, Docker images for the API and frontend are built.

4. On push events, the images are pushed to Docker Hub with branch, SHA, and latest tags on the default branch.

5. On main, the deploy job runs and still documents the reproducible local-first production path through docker-compose.prod.yml and scripts/deploy.sh.

6. The production profile pulls published images instead of rebuilding them, which demonstrates reproducible deployment using the same service topology as development.

## Key Decisions and Trade-offs

**Static frontend:** A static frontend keeps the focus on infrastructure while still showing reverse proxying, container separation, and frontend-to-API traffic.

**Spring Boot API:** Spring Boot provides a realistic backend shape with health checks, PostgreSQL integration, and a familiar build pipeline without unnecessary application complexity.

**Docker Hub registry:** Docker Hub public images reduce friction for reviewers because they can inspect and pull images immediately.

**GitHub Actions:** GitHub Actions keeps the pipeline close to the repository and clearly expresses the stages: test, build, publish, and deploy trigger.

**Uptime Kuma:** Uptime Kuma is lightweight and easy to demonstrate locally. It directly answers whether the frontend, API, and database are available.

**Secrets handling:** Secrets stay in local .env files or GitHub Actions secrets, while .env.example documents the required variables without exposing real credentials.

**Iheb Belaid**
+216 95 827 073 | ihebbelaid3@gmail.com | linkedin.com/in/iheb-belaid/

## Dev / Prod Parity

Development and production use the same core runtime shape: Nginx frontend, Spring Boot API, PostgreSQL, and Uptime Kuma. The main difference is image source.

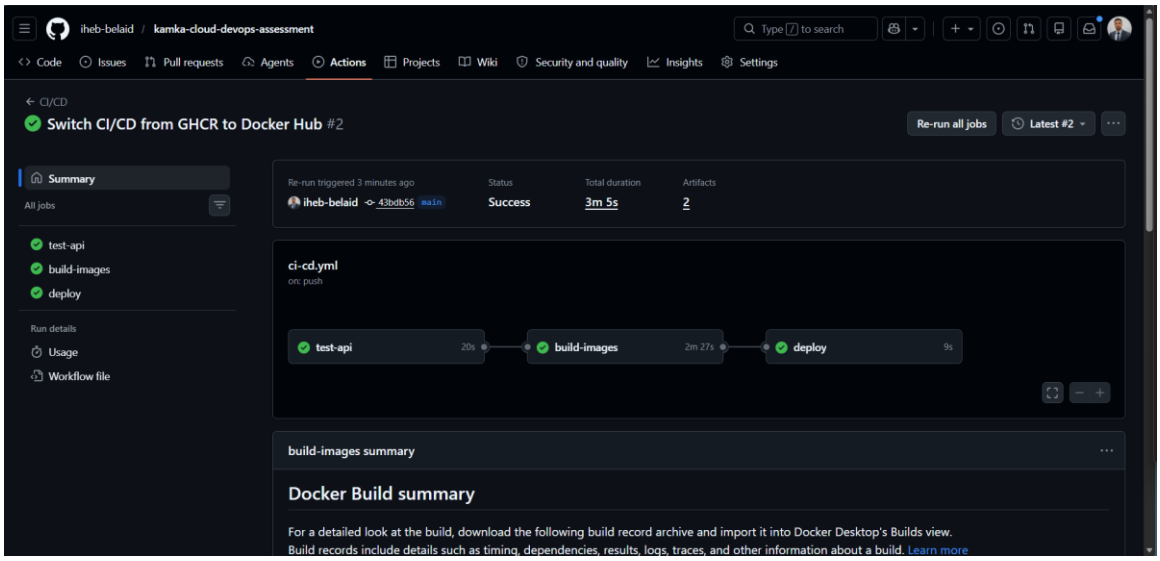
- development builds images locally with docker compose up --build
- production pulls versioned images from Docker Hub with docker-compose.prod.yml
- health checks, service dependencies, and monitoring stay consistent across both environments

## Validation Performed

- docker compose up --build -d starts the development stack successfully
- docker compose -f docker-compose.prod.yml --env-file .env up -d starts the production profile successfully
- the frontend is reachable on http://localhost:3000
- Spring Boot health responds on http://localhost:8081/actuator/health
- PostgreSQL accepts connections
- Uptime Kuma is reachable on http://localhost:3001 and monitors frontend, API, and PostgreSQL
- GitHub Actions completed successfully for test, build, image push, and deploy-stage signaling
- Docker Hub contains public repositories for kamka-todo-api and kamka-todo-frontend

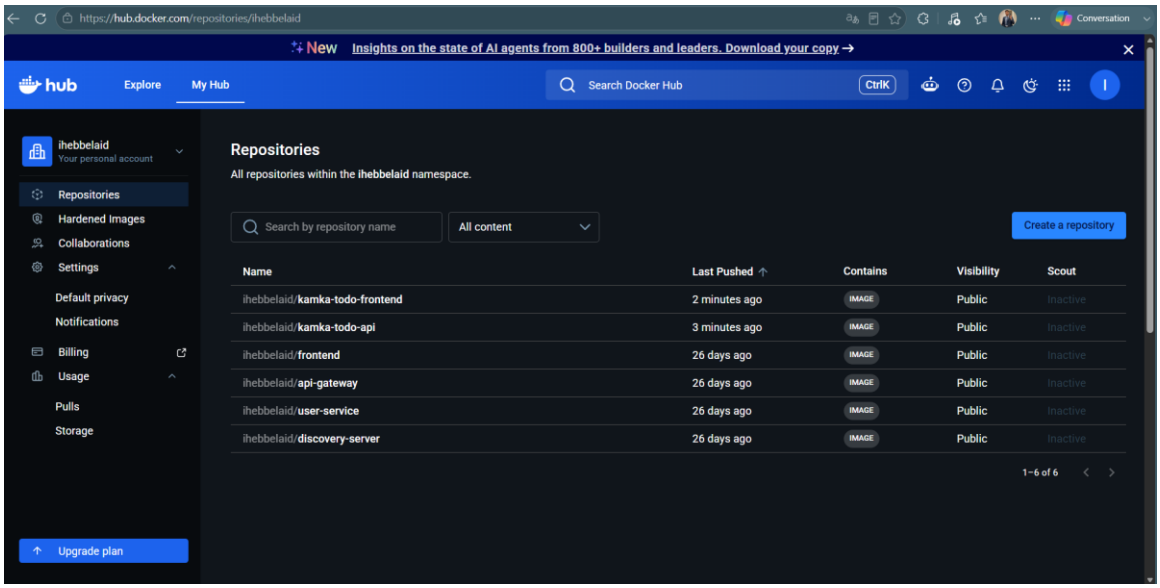
## Validation Screenshots

The most relevant validation screenshots are embedded below because the submission form does not provide separate screenshot upload slots.

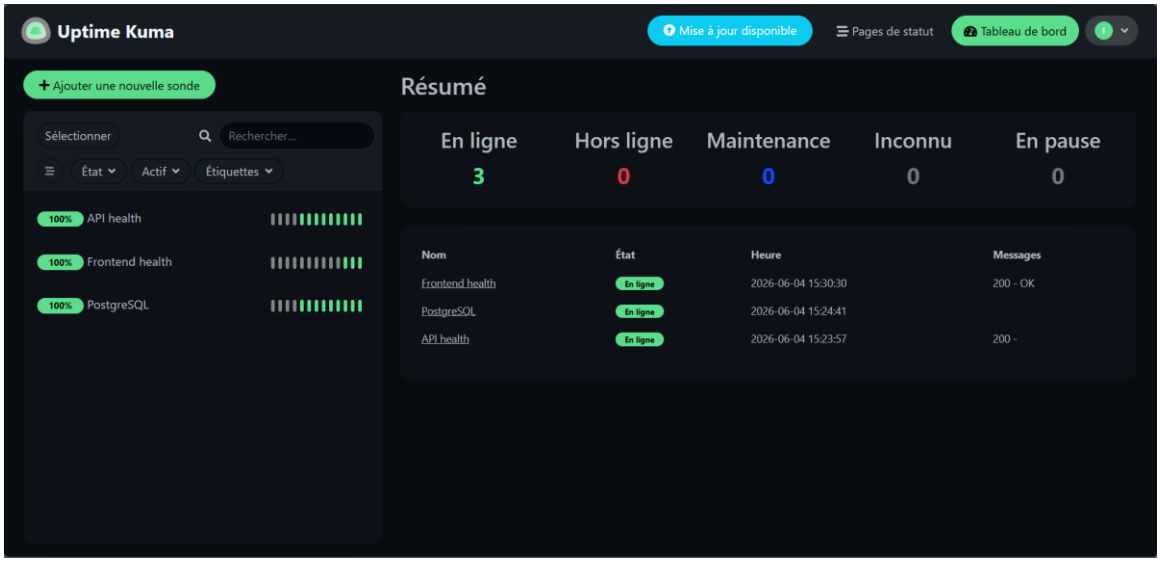



*GitHub Actions workflow completed successfully*

**Iheb Belaid**
+216 95 827 073 | ihebbelaid3@gmail.com | linkedin.com/in/iheb-belaid/

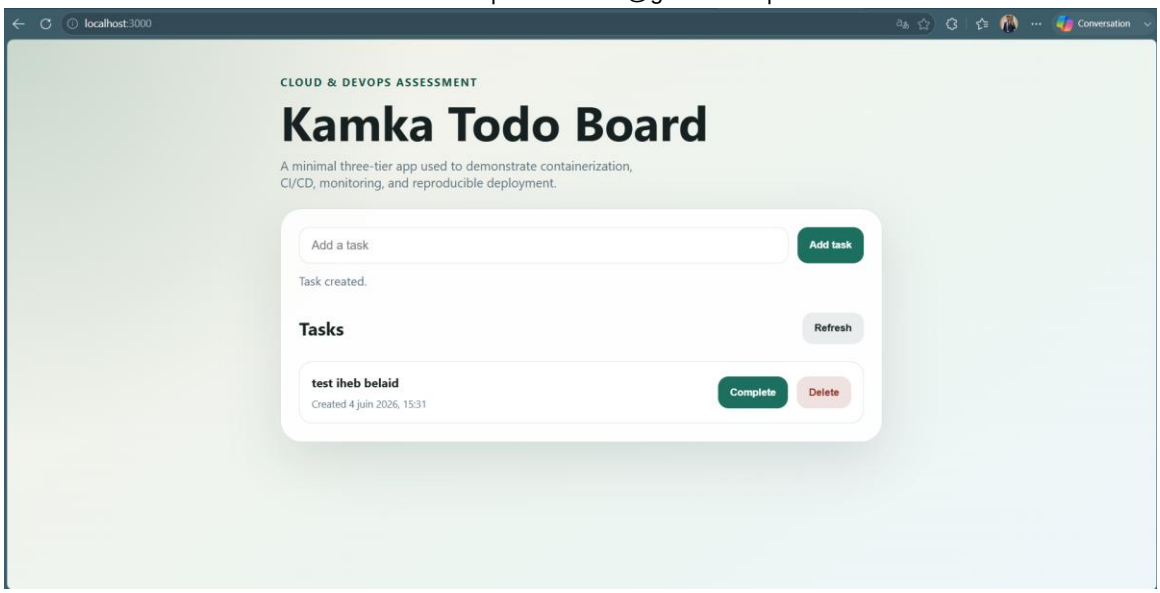



*Docker Hub repositories published successfully*

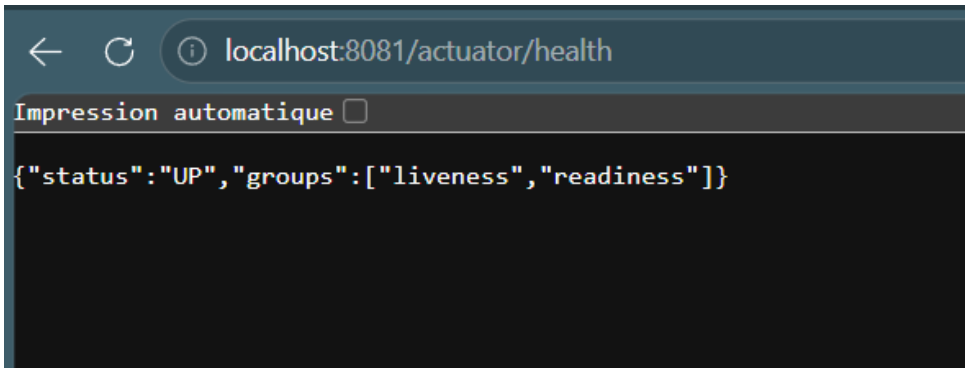



*Uptime Kuma monitoring the full stack with three healthy checks*

**Iheb Belaid**
+216 95 827 073 | ihebbelaid3@gmail.com | linkedin.com/in/iheb-belaid/




*Frontend application running locally*




*Spring Boot health endpoint responding with UP status*

## Known Limitations

- Uptime Kuma monitors are created manually on first boot rather than auto-provisioned.
- A visible public live host is not provisioned yet; the deploy job currently demonstrates reproducible deployment plus optional remote execution.
- HTTPS was left out because it is optional once the core requirements are complete.
- Rollback is manual through previously published image tags rather than automated.
- The application stays intentionally minimal and does not include authentication or advanced data migration handling.

## What I Would Improve With More Time

- Provision a small public Linux host and complete a live deployment..
- Introduce a backup script and recovery notes for PostgreSQL.
- Add release-tag-based rollback automation.
- Add infrastructure automation with Terraform or Ansible if the scope expands.